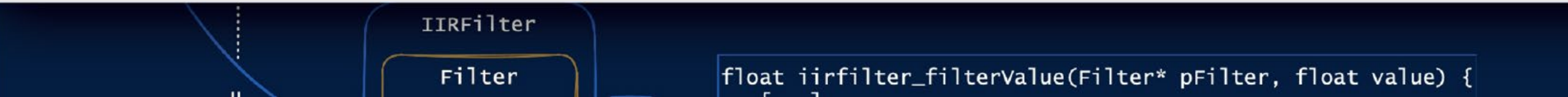
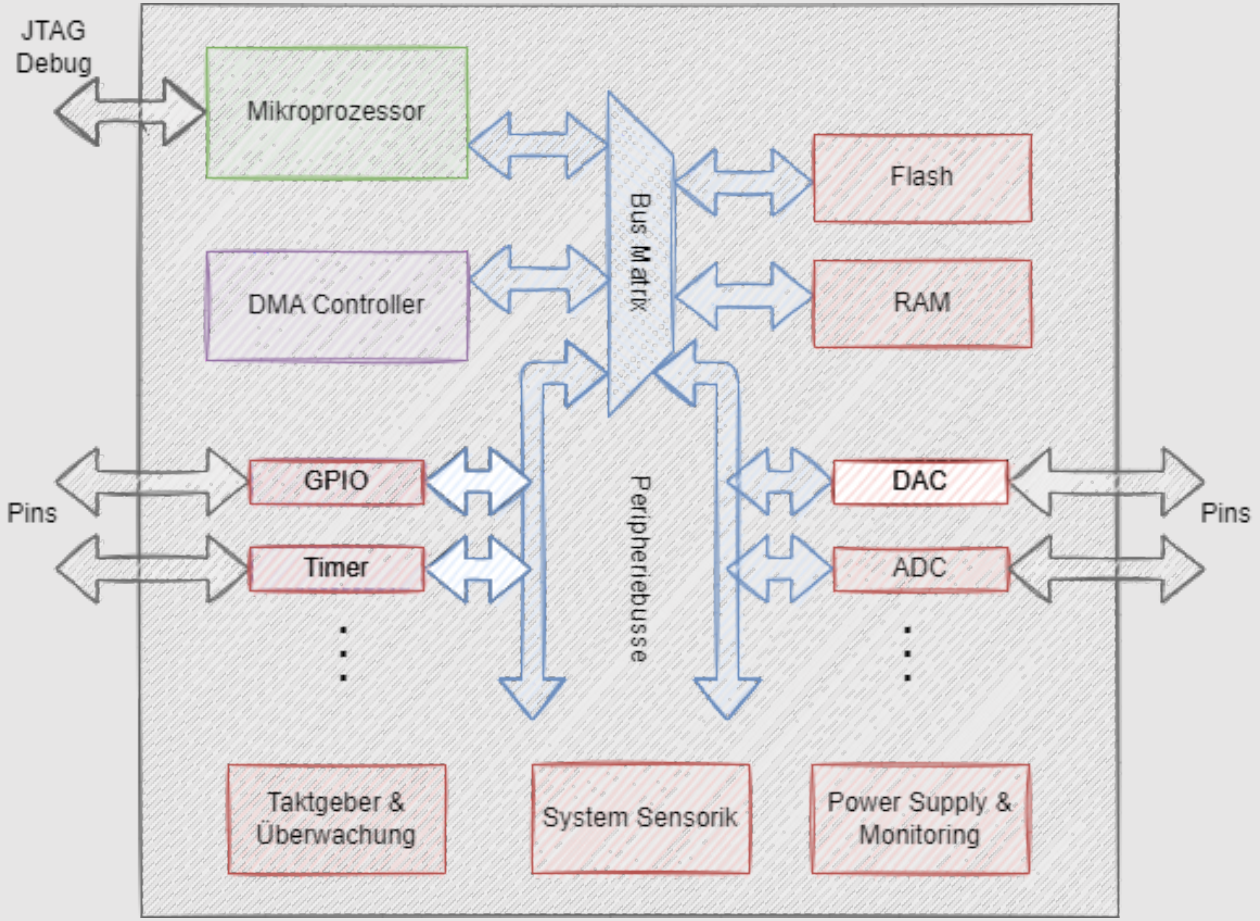
Embedded Systems mit RISC-V

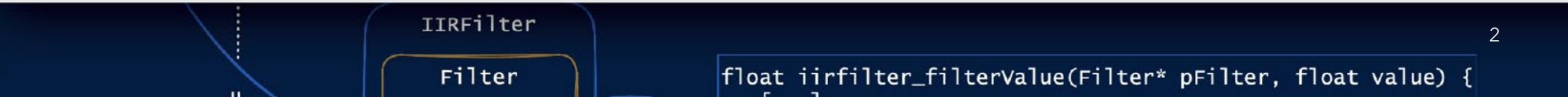
4. Der Mikrocontroller

V1.0, Patrick Ritschel

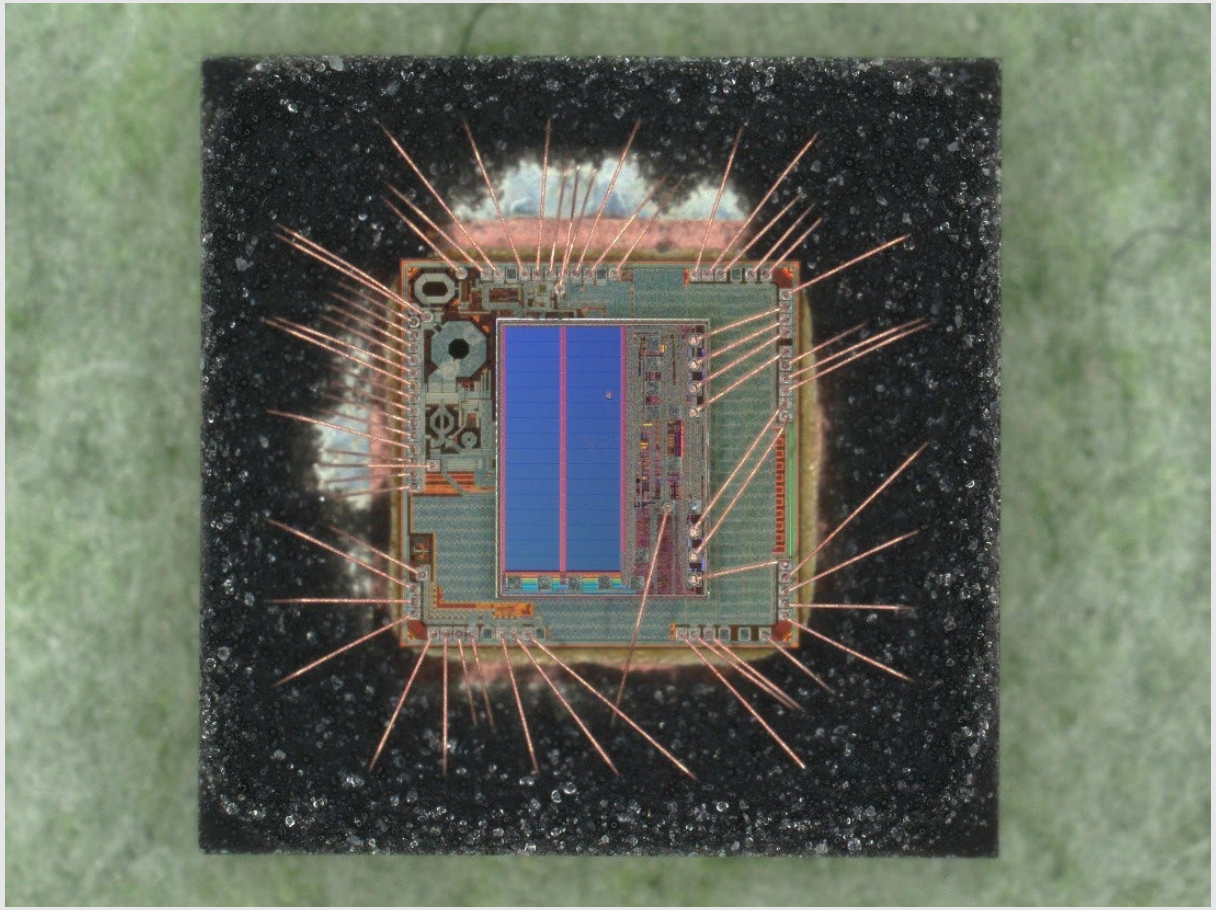


Der Mikrocontroller
Schematischer Aufbau im Blockschaltbild





Der Mikrocontroller
Silizium-Die des ESP32-C3

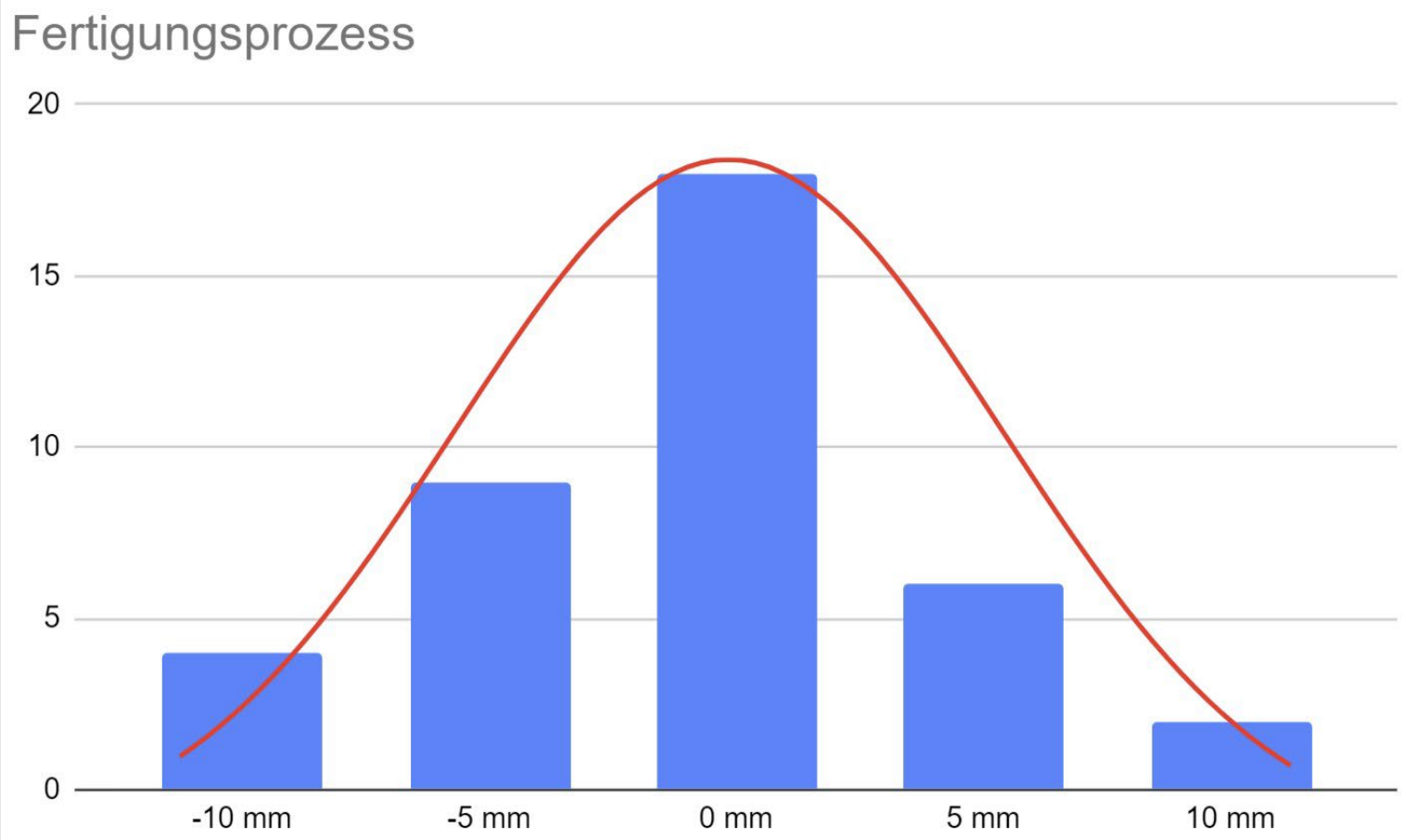


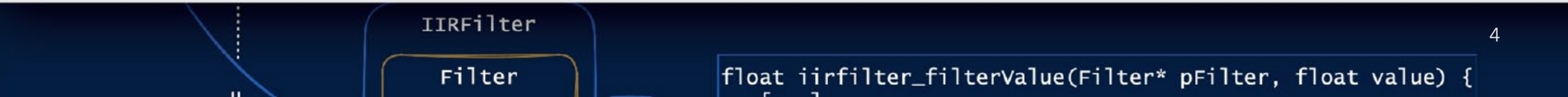
IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {
```

Normalverteilung
Werkstücke in einem Fertigungsprozess

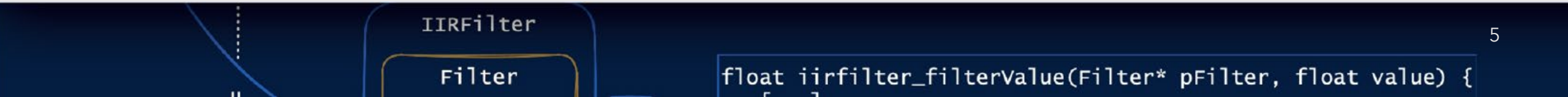




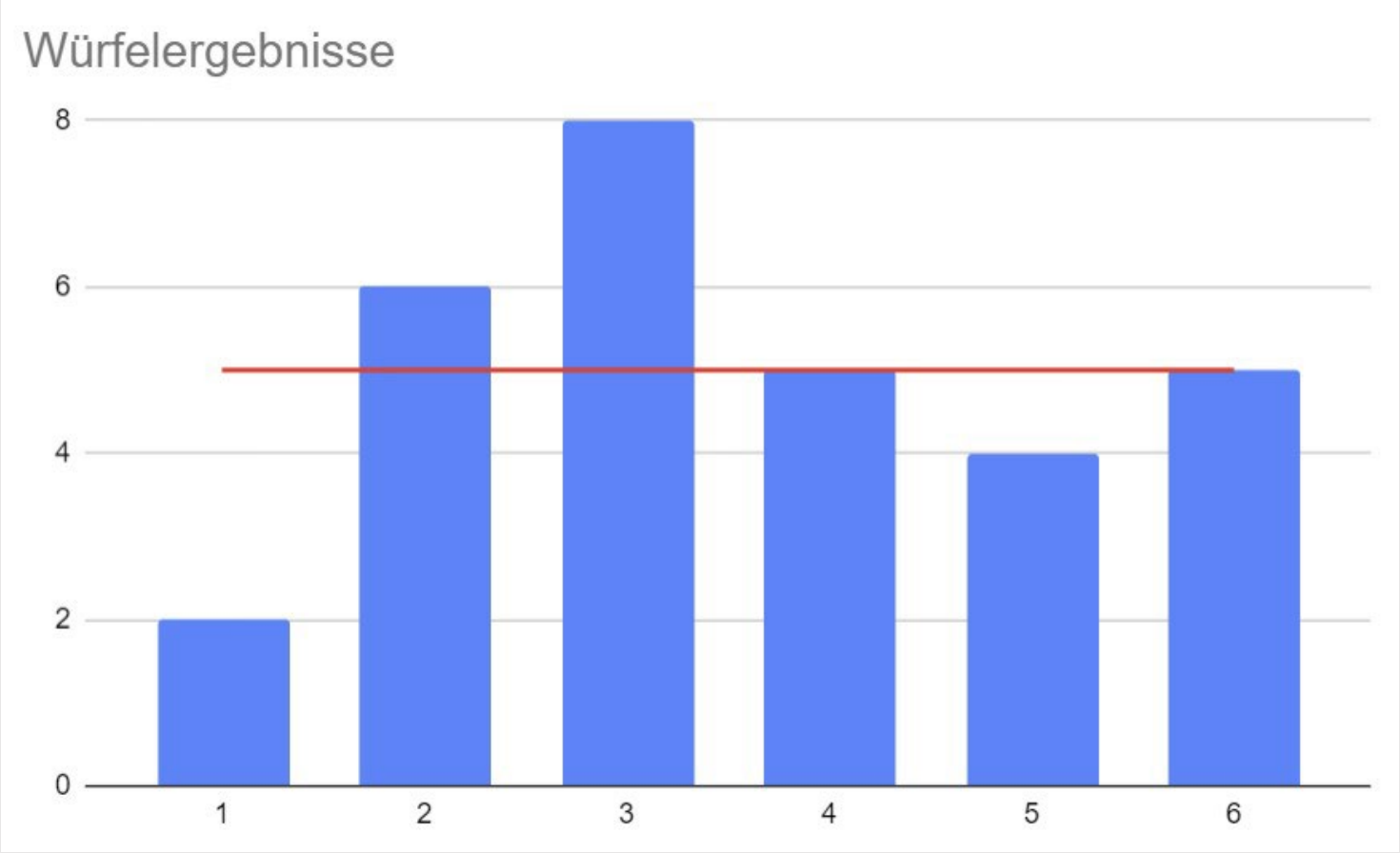
Normalverteilung
Chi-Quadrat Test

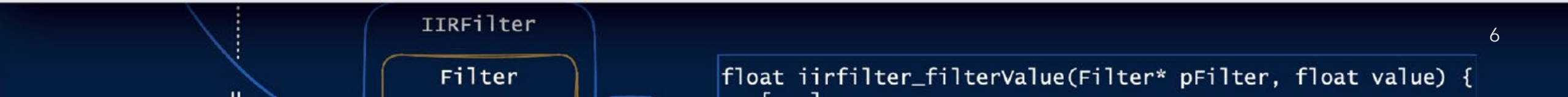
$$X^2 = \sum_{i=1}^{m} \frac{(N_i - n_{0i})^2}{n_{0i}}$$

Die »Nullhypothese« H_0, die besagt, dass X eine zu überprüfende Verteilung aufweist, wird bei einem Signifikanzniveau α abgelehnt, wenn $X^2 > \chi^2_{(1-\alpha;m-1)}$. Die χ^2-Quantile mit entsprechenden Freiheitsgraden $m - 1$ werden üblicherweise einer Tabelle entnommen



Gleichverteilung
Würfelergebnisse bei 30 Würfen



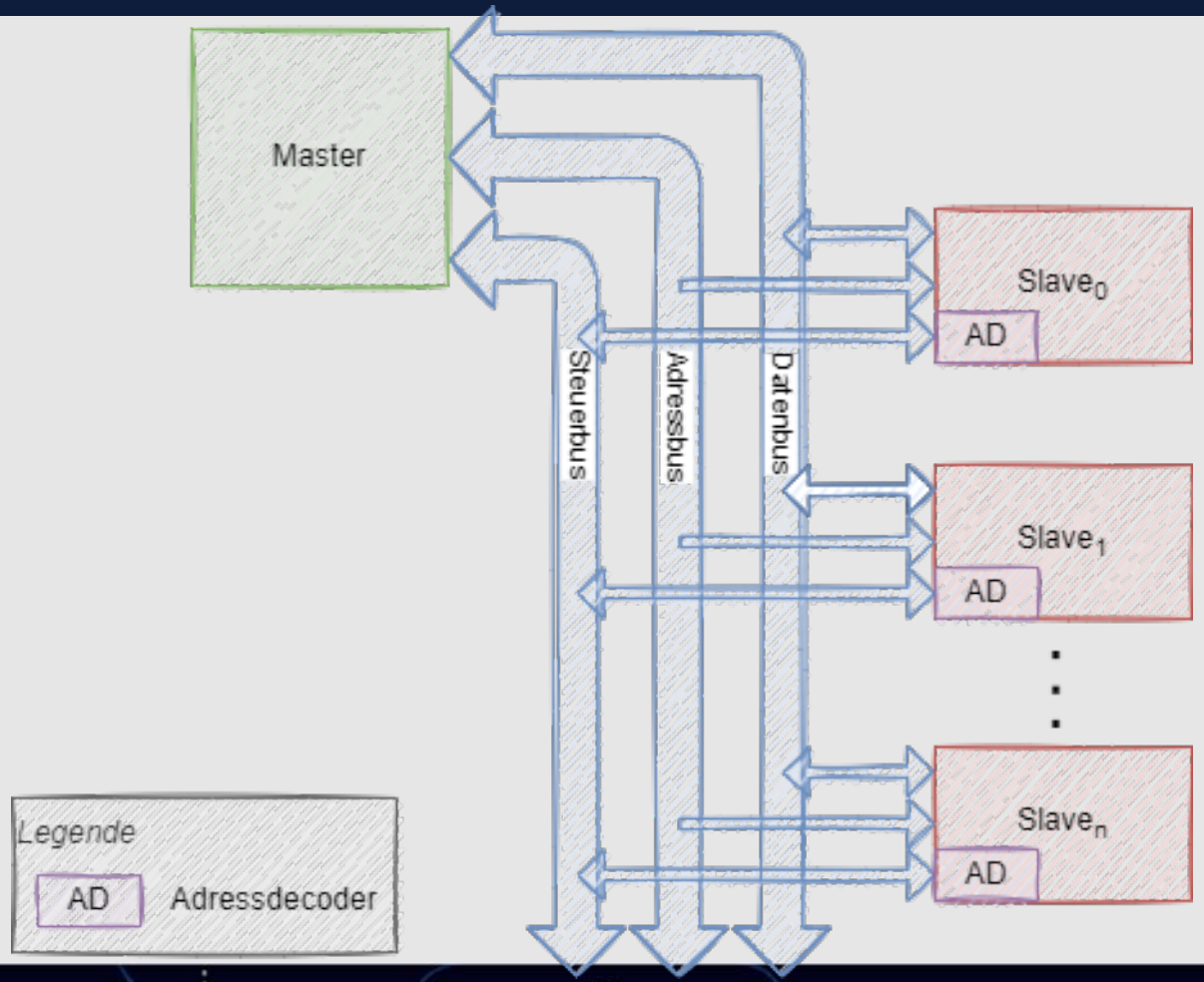


Chi-Quadrat Test für Gleichverteilung
Implementierung mit Vereinfachung


```
#define M                   6
#define OBSERVATIONS        30
#define SQUARE(x)           ((x) * (x))
static const uint32_t N[M] = { 2, 6, 8, 5, 4, 5 };

bool equalDistChi2(const uint32_t n[], uint32_t m,
    ↪  uint32_t n0, uint32_t chi2) {
    uint32_t squaresum = 0;
    for (int i = 0; i < m; i += 1) {
        squaresum += SQUARE(n[i] - n0);
    }
    uint32_t x2 = squaresum / n0;
    return (x2 <= chi2);
}

bool ok = equalDistChi2(N, M, (OBSERVATIONS / M), 9); // Aufruf
```

$$\forall_i : n_{0i} = n_0 \Rightarrow X^2 = \sum_{i=1}^{m} \frac{(N_i - n_0)^2}{n_0} = \frac{\sum_{i=1}^{m}(N_i - n_0)^2}{n_0}$$

Der Mikrocontroller
Paralleles Bussystem



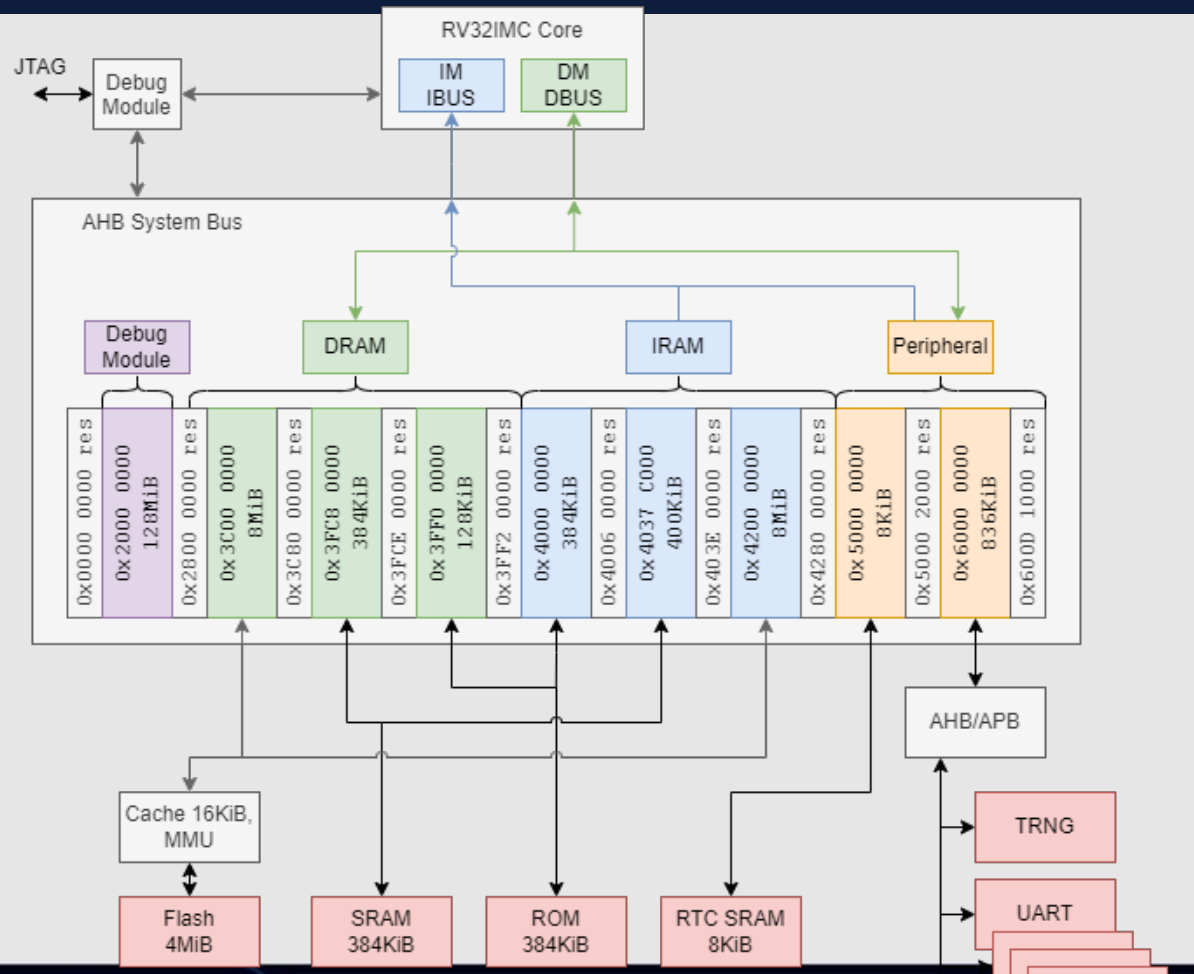
Modul	Adressbereich
$Slave_0$	0x00010000 - 0x0001FFFF
$Slave_1$	0x00020000 - 0x00020FFF
$Slave_2$	0x00040300 - 0x000403FF

IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {
```

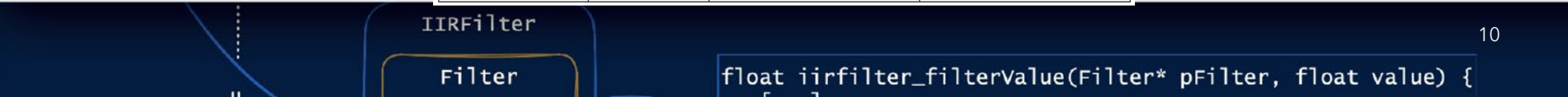
ESP32-C3
Adress- und Bussystem



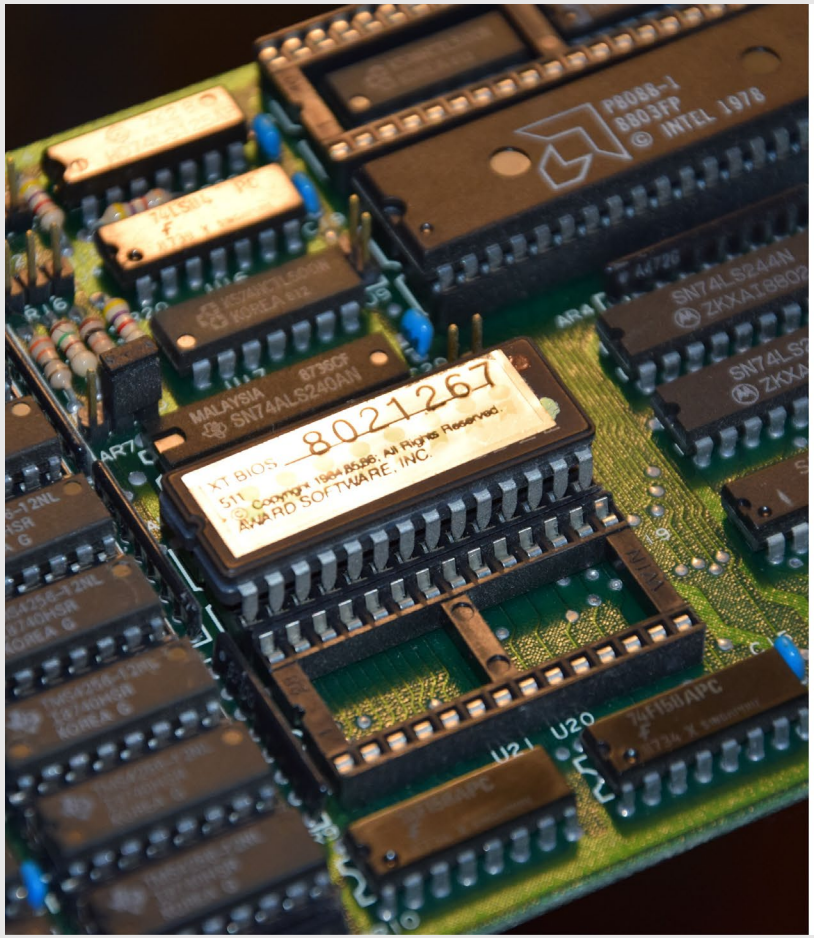

```
IIRFilter
Filter
float iirfilter_filterValue(Filter* pFilter, float value) {
```

Speichertechnologien
Vor- und Nachteile

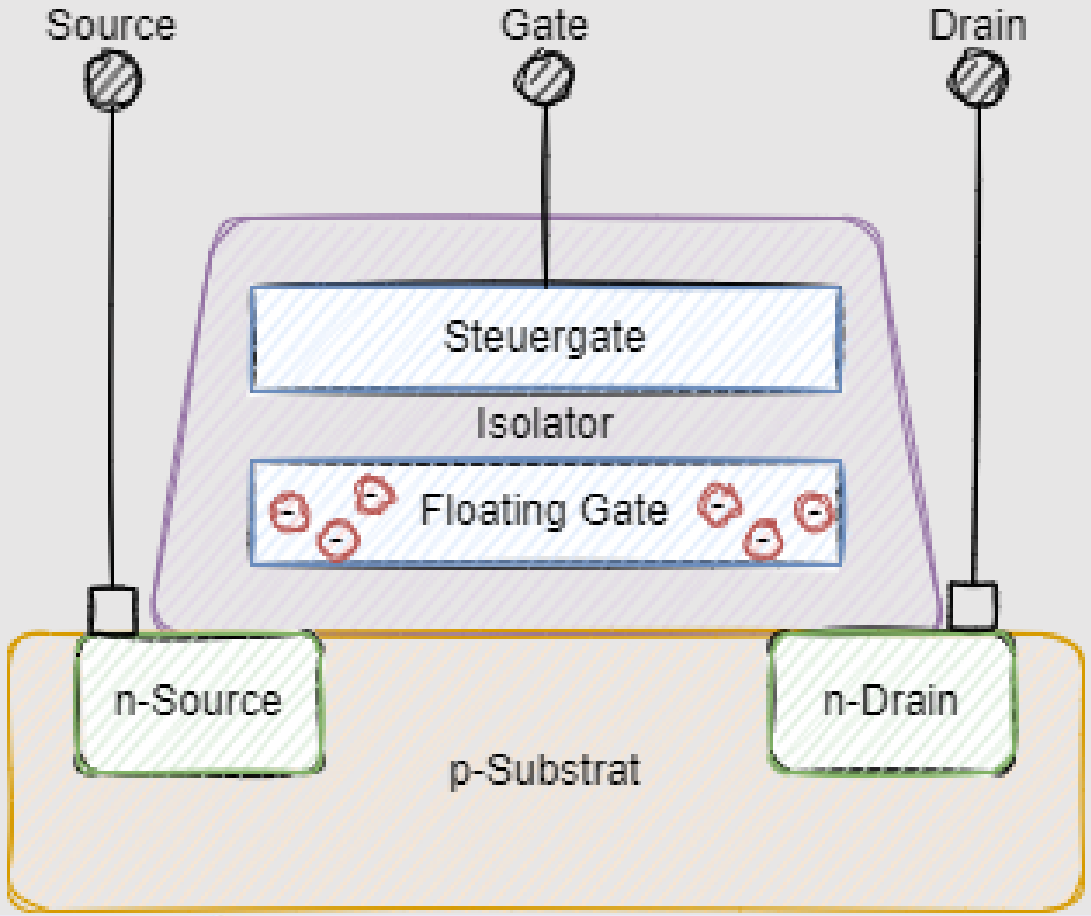
Technologie	Persistenz	Vorteile	Nachteile
SRAM	volatil	schnell, stromsparend	teuer
DRAM	volatil	günstig	langsam, stromhungrig
Masken-ROM	non-volatil	am günstigsten, schnell	Änderungen unmöglich
PROM	non-volatil	schnell lesbar	einmalig beschreibbar, teuer
EPROM	non-volatil	löschbar	Programmer und Löschgerät nötig
EEPROM	non-volatil	vielfach schreibbar	begrenzte Schreib-/Löschzyklen, langsam
FRAM	non-volatil	schnell, RAM-Ersatz, lange haltbar	sehr teuer
ReRAM	non-volatil	stromsparend	begrenzte Schreibzyklen



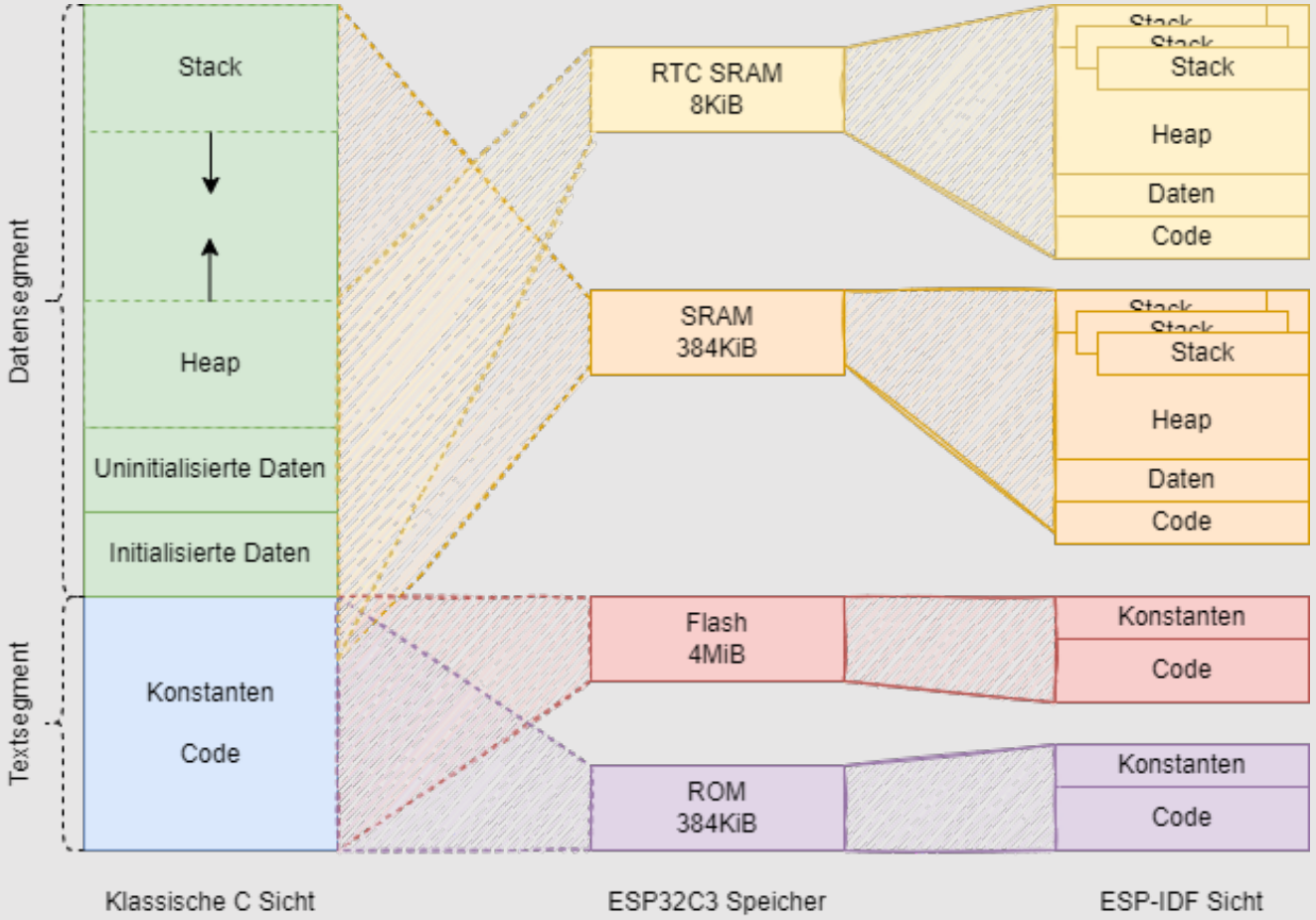
Speichertechnologien
EPROM

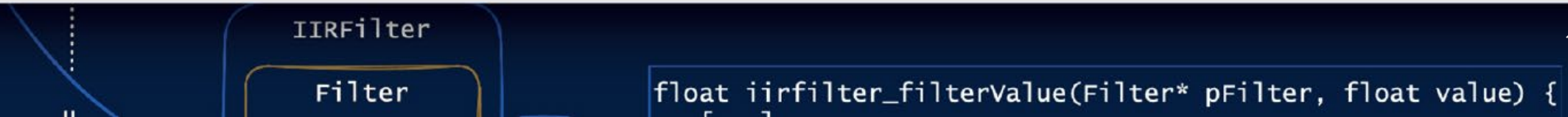


Speichertechnologien
Aufbau (E)EPROM



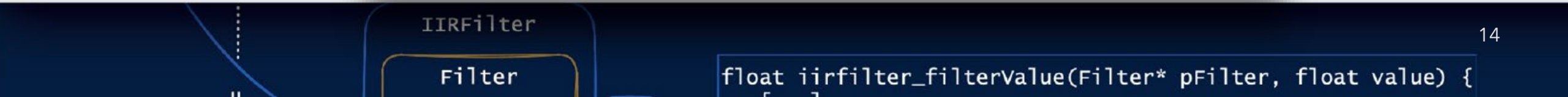
Speicher
Zuordnung der Applikationssegmente





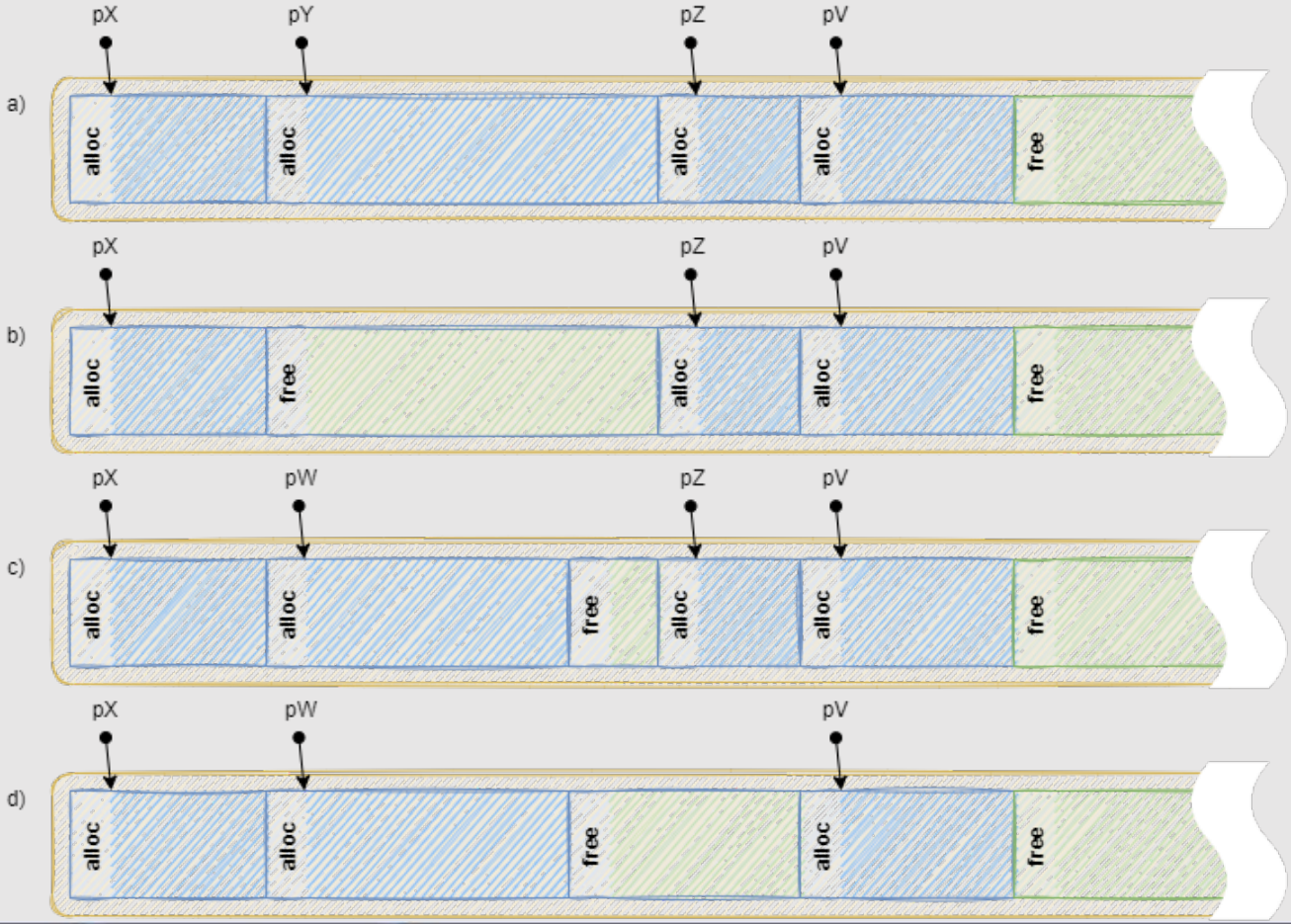
Speicher
Statischer vs. dynamischer Speicher

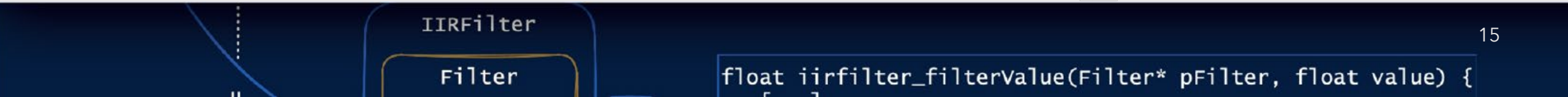
Speicherung	Vor-/Nachteile
statisch	△ Speicherprobleme zur Compile-Zeit △ deterministisch ▽ Maximum an Speicher notwendig
dynamisch	△ speichereffizient ▽ Speicherprobleme zur Laufzeit ▽ Fragmentierung ▽ indeterministische Dauer bei Allokation und Freigabe



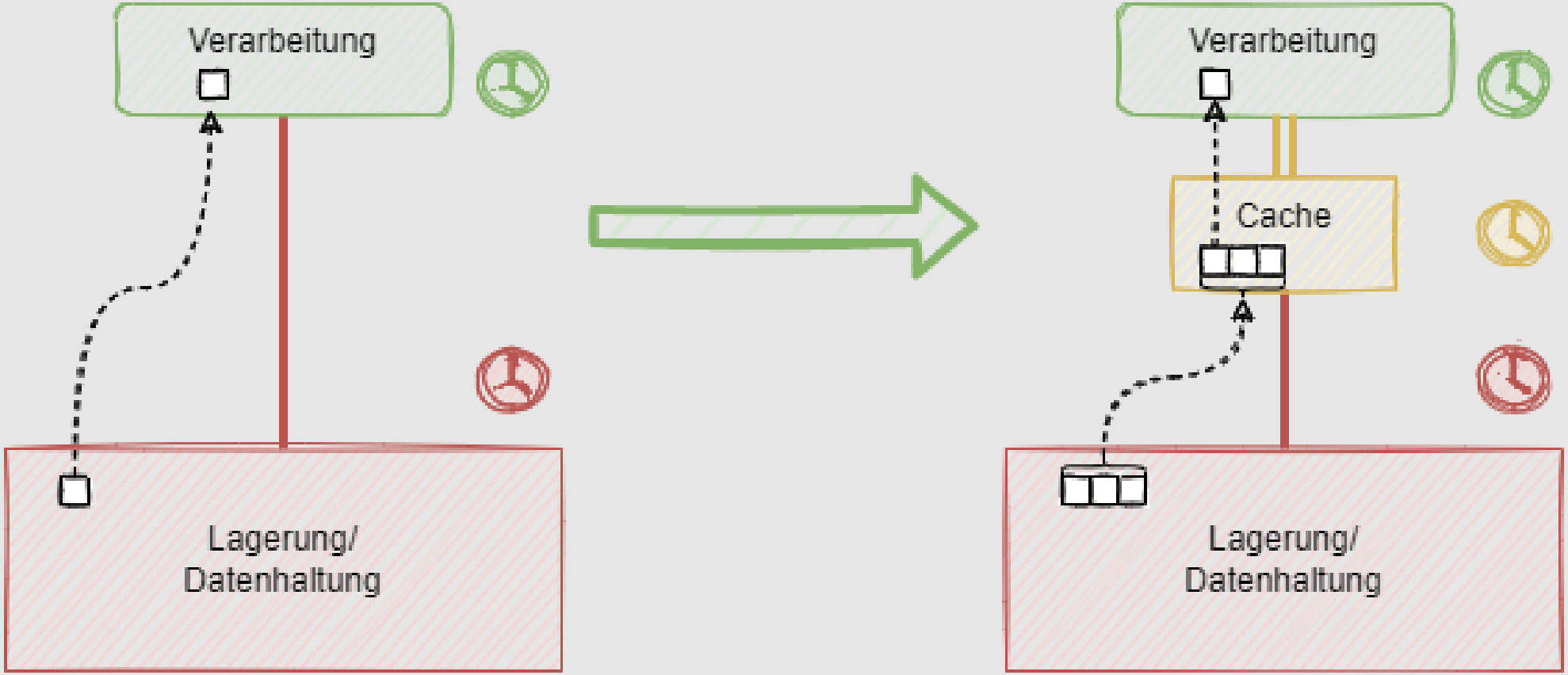
Dynamischer Speicher

a) Blockaufbau - b) nach `free(pY)` - c) nach `malloc()` - d) nach `free(pZ)`

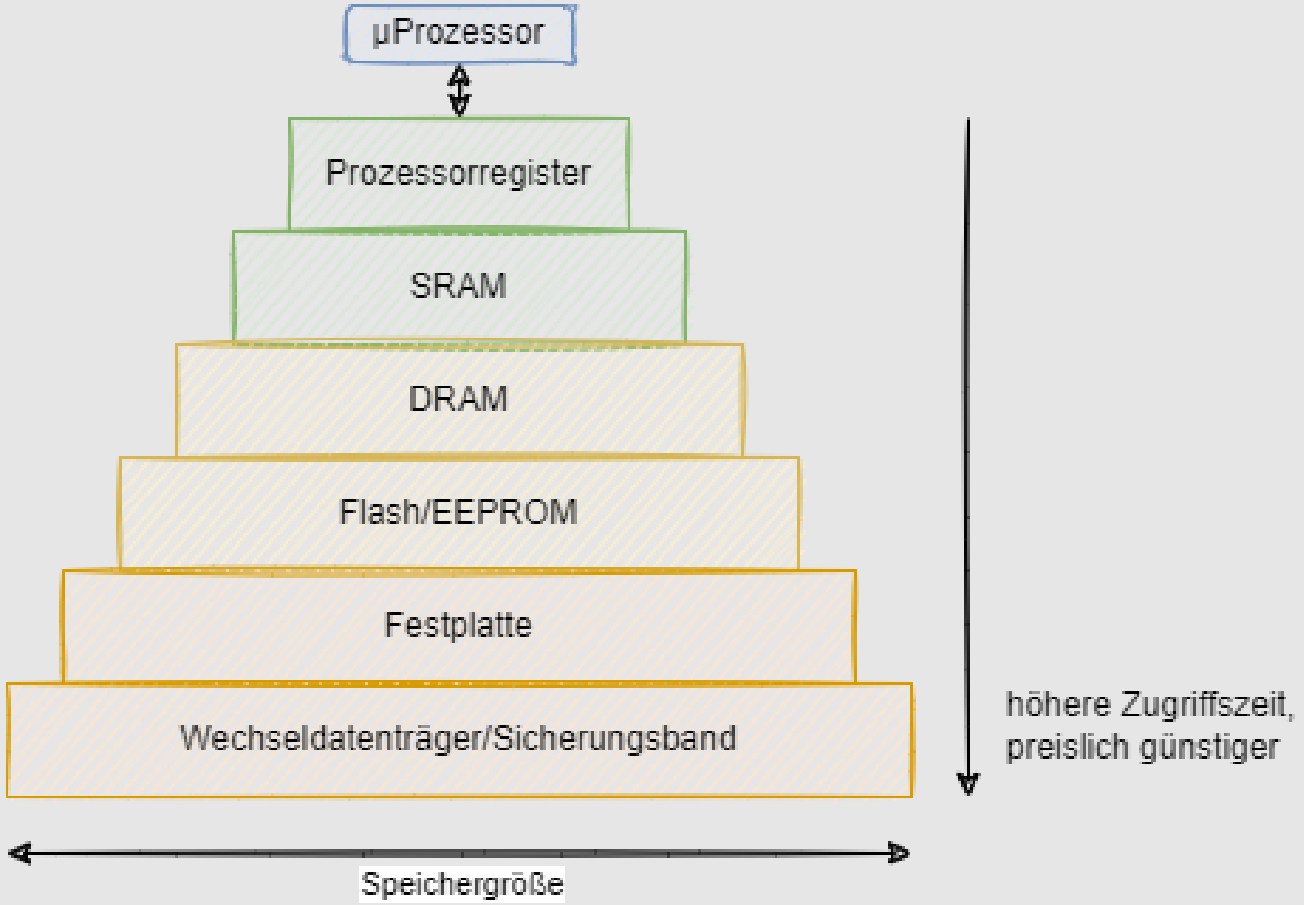


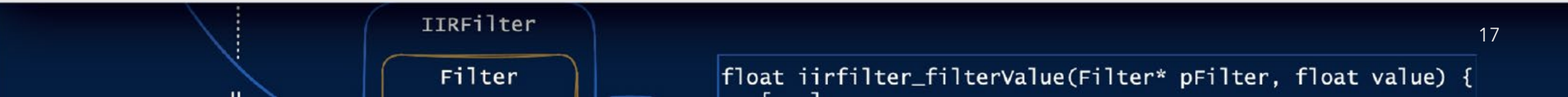


Speicher
Cache zur Performanzsteigerung

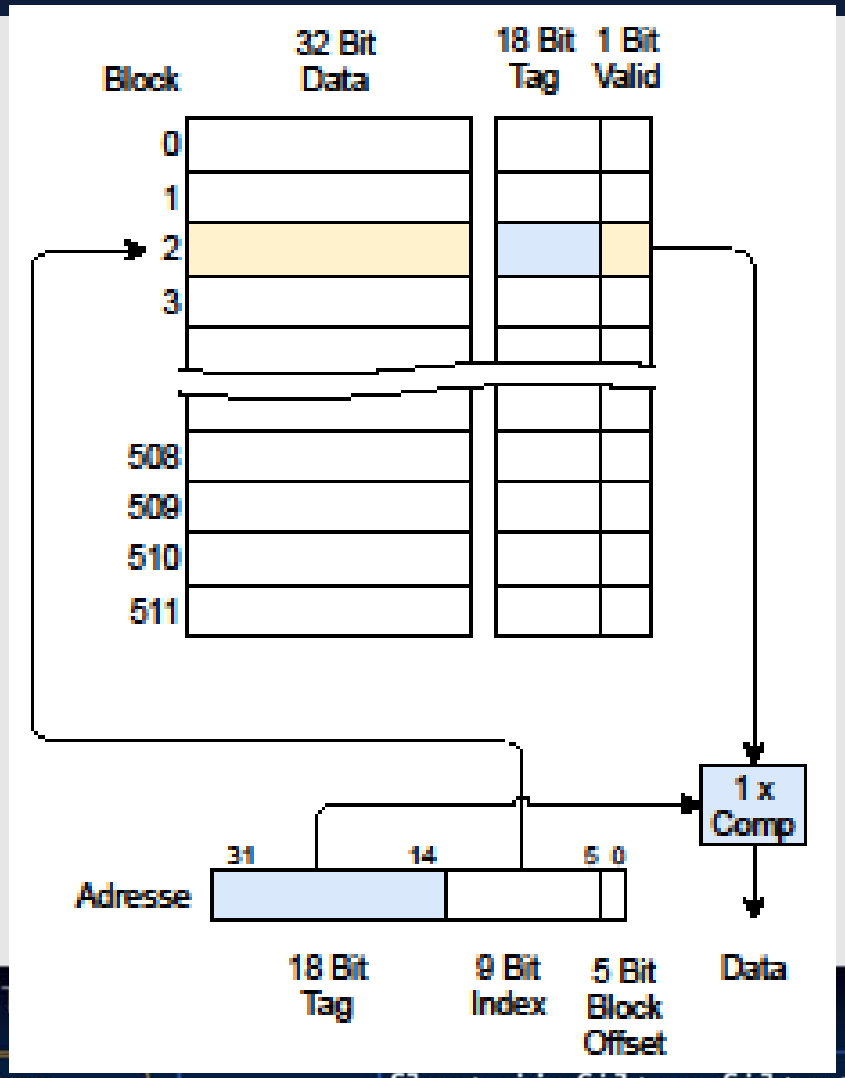


Speicherhierarchie von PC-Systemen

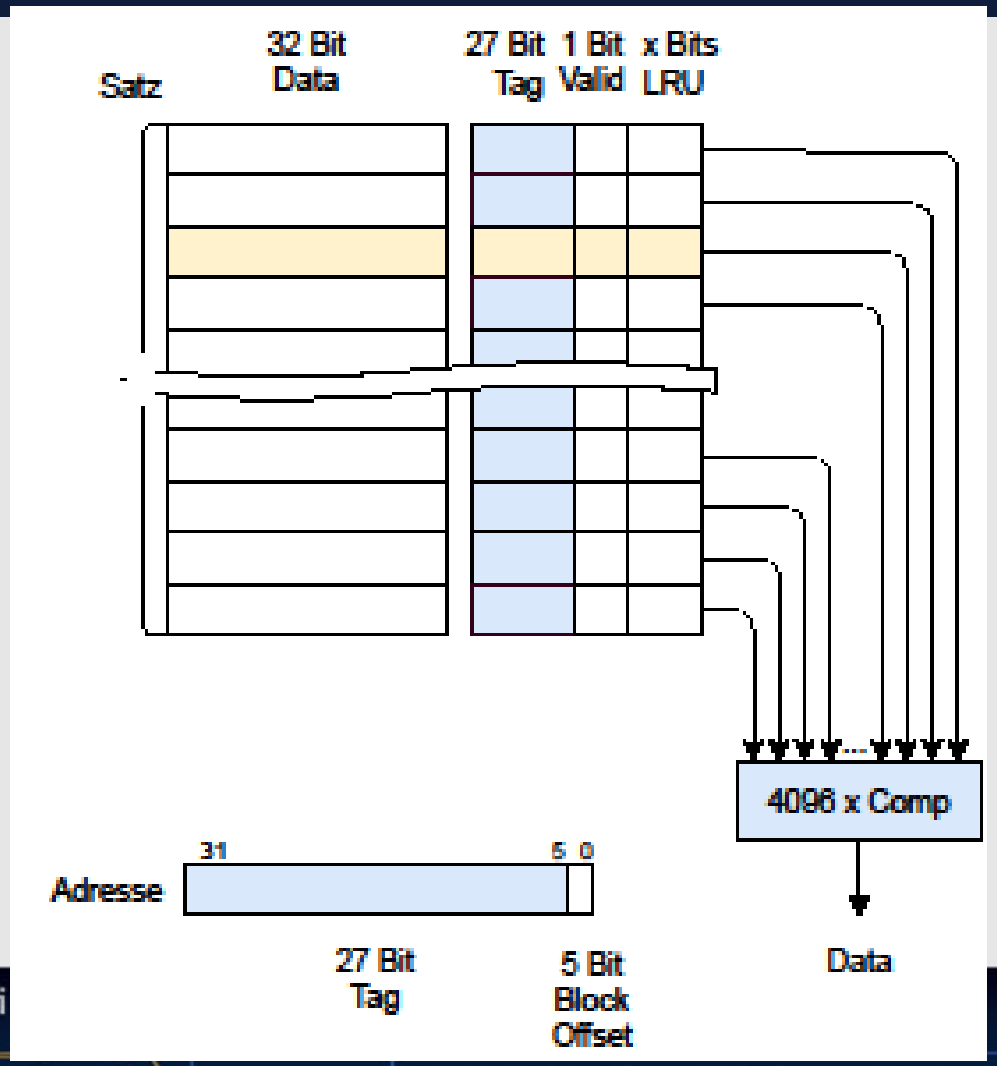




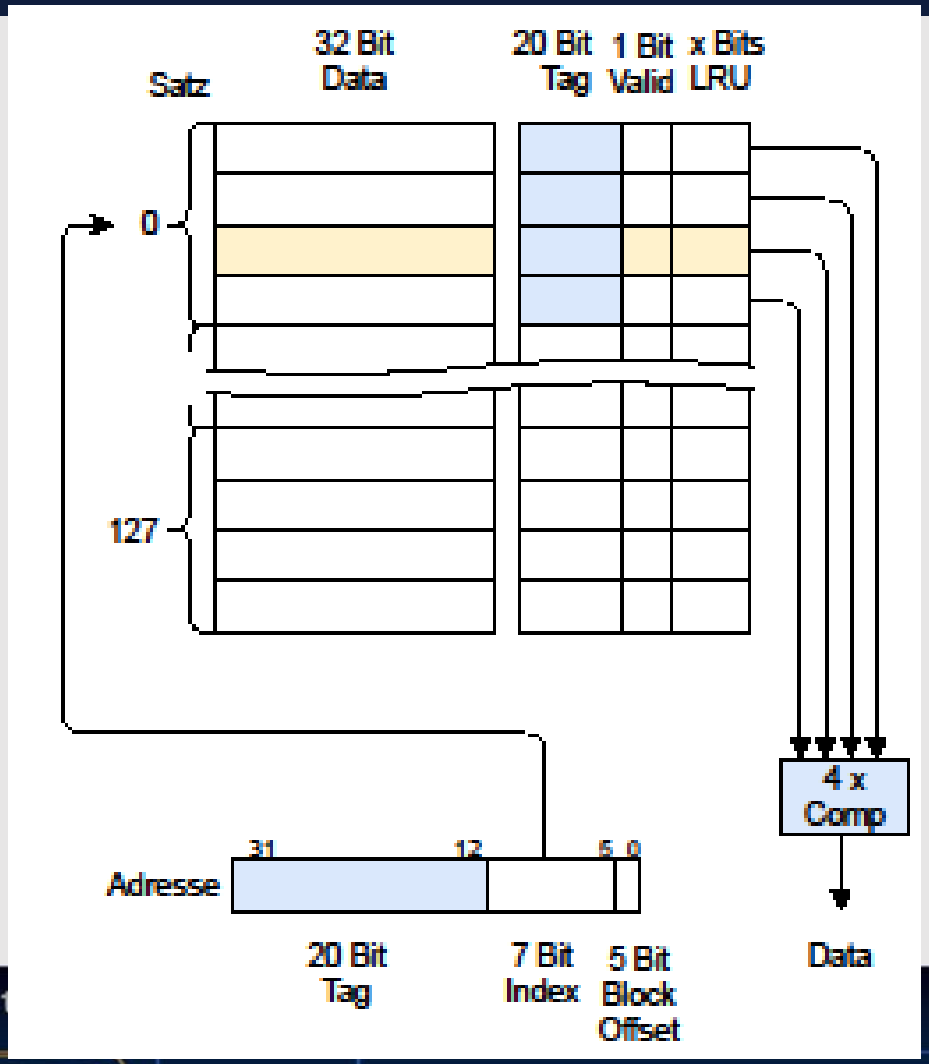
Cache-Organisation
16-KiB direct-mapped Cache mit 32 B-Blöcken



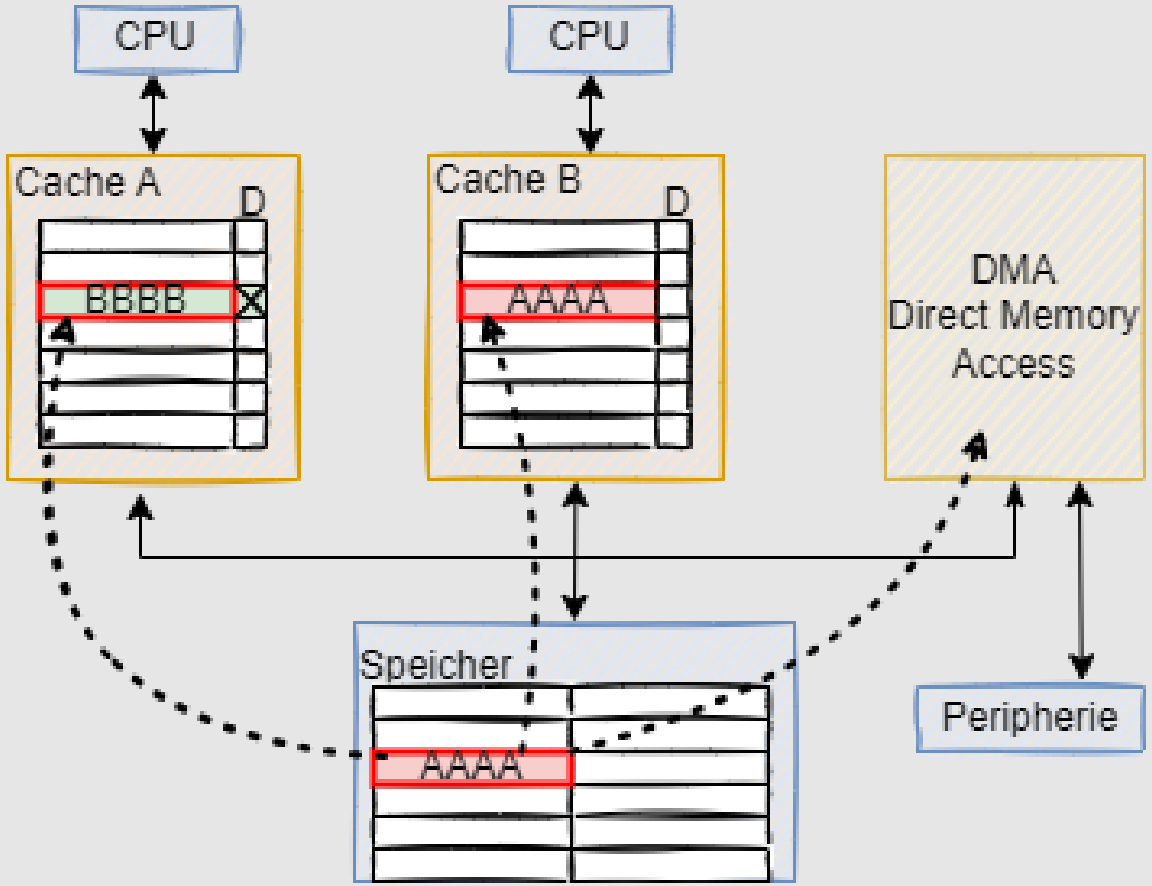
Cache-Organisation
16-KiB fully-associative Cache



Cache-Organisation
16-KiB 4-way set-associative Cache



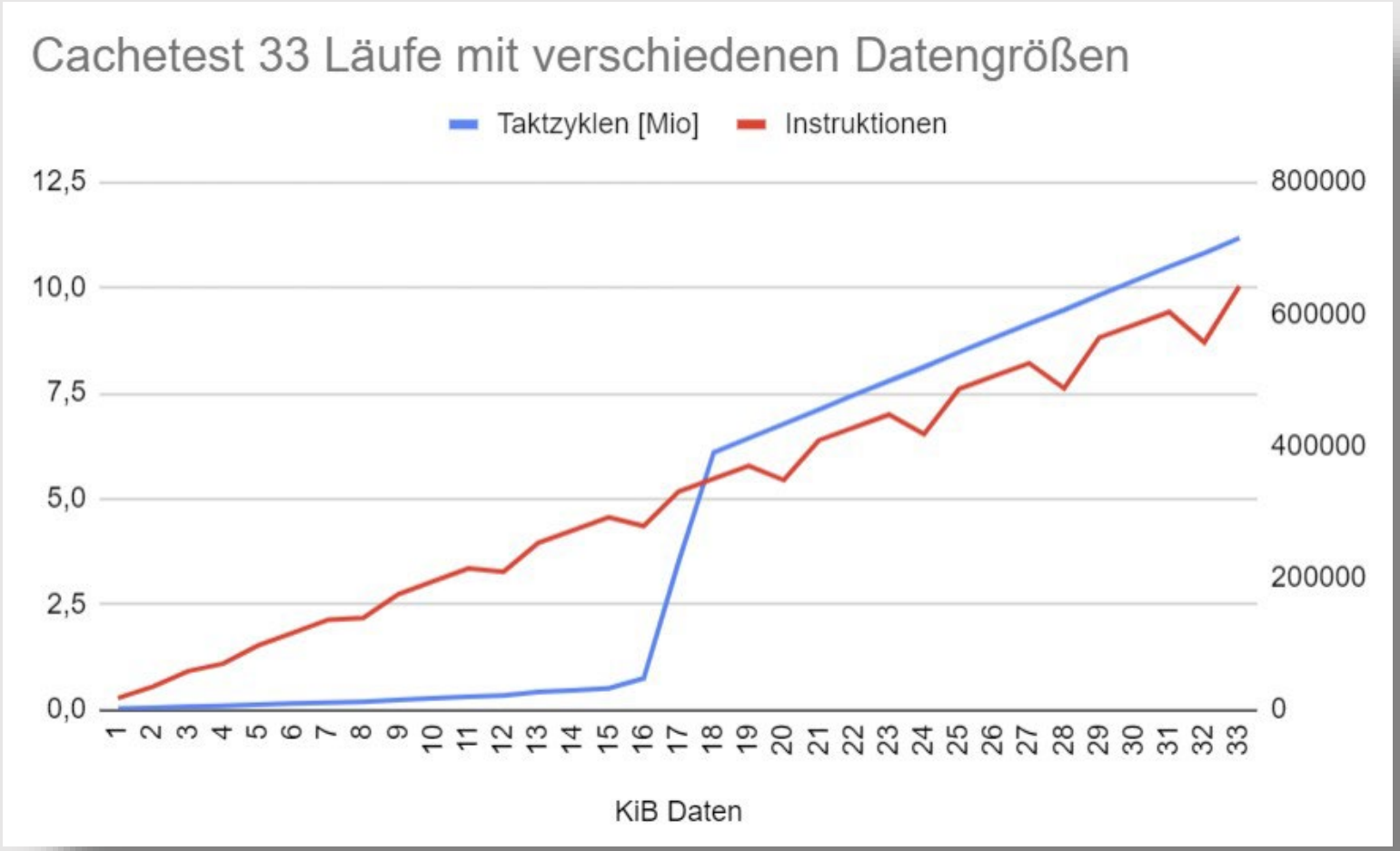
Cache Konsistenz



Cache
Test mit Histogrammapplikation


```
#define TESTSIZE                (1 * 1024)
#define BLOCKSIZE               32
const char gNibelungenlied[]=
    "Aventivre von den nibelvngen: Vns ist in alten "
    "maeren wunders vil geseit von heleden lobebaeren "
    "von grozer arebeit [...]";
// [...]
uint32_t histogram[256] = { 0 };
uint32_t offs = 0;
for (uint32_t i = 0; i < TESTSIZE; i += 1) {
    histogram[(unsigned char)gNibelungenlied[offs]] += 1;
    offs += BLOCKSIZE;
    if (offs >= TESTSIZE) {
        offs -= TESTSIZE;
        offs += 1;
    }
}
// [...]
```

Cache
Messergebnisse der Histogrammapplikation



Der Linker
Platzierung in Speichersegmenten


```
__attribute__((section(".noinit"))) uint32_t gKeepValue;
```
==
```
__NOINIT_ATTR uint32_t gKeepValue2;
```

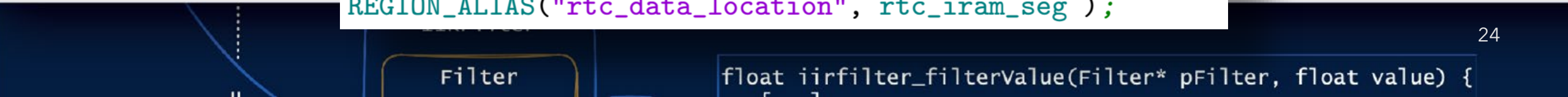
```
RTC_NOINIT_ATTR uint32_t gDeepKeepValue;
```

Map-File

```
.rtc_noinit   0x50000010   0x4                main.c.obj
              0x50000010   gDeepKeepValue
```

Linker Script

```
MEMORY
{
  rtc_iram_seg(RWX) : org = 0x50000000, len = 0x2000 - 0
}
REGION_ALIAS("rtc_data_location", rtc_iram_seg );
```

Der Linker
Alignment und Zugriff auf Linker-Variablen

Alignment setzen

```
.rtc_noinit (NOLOAD):
{
  . = ALIGN(4);
  _rtc_noinit_start = ABSOLUTE(.);
  *(.rtc_noinit .rtc_noinit.*)
  . = ALIGN(4) ;
  _rtc_noinit_end = ABSOLUTE(.);
} > rtc_data_location
```

Zugriff auf Linker-Variablen

```
extern uint32_t _rtc_noinit_start;
extern uint32_t _rtc_noinit_end;
```

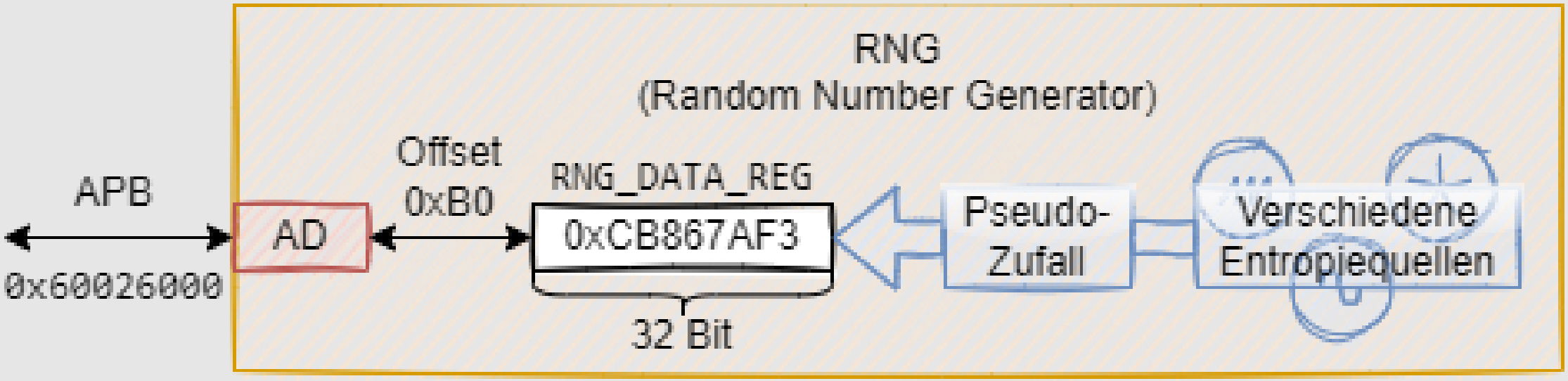
```
uint32_t* pRTCNoinitStart = &_rtc_noinit_start;
uint32_t* pRTCNoinitEnd = &_rtc_noinit_end;
printf("rtc noinit start 0x%8p, rtc noinit end 0x%8p\n",
    ↪ pRTCNoinitStart, pRTCNoinitEnd);
```

IIRFilte

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {
```

Peripheriezugriff
auf den Zufallszahlengenerator



Peripheriezugriff
auf den Zufallszahlengenerator

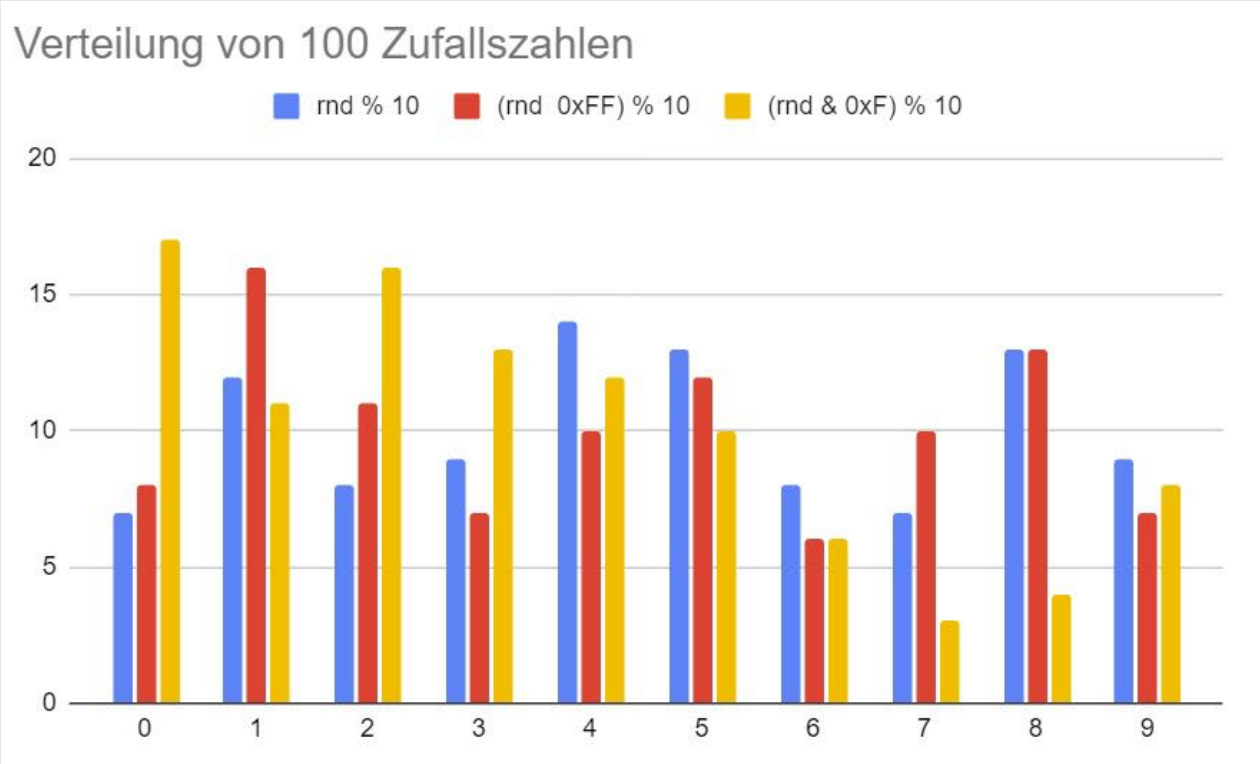

```
#define RNG_BASE 0x60026000
#define RNG_DATA_REG_OFFS 0xB0

volatile uint32_t* pRngDataReg =
    ↪ (volatile uint32_t*)
    ↪ (RNG_BASE | RNG_DATA_REG_OFFS);

inline uint32_t nextRand() {
    return *pRngDataReg;
}
```

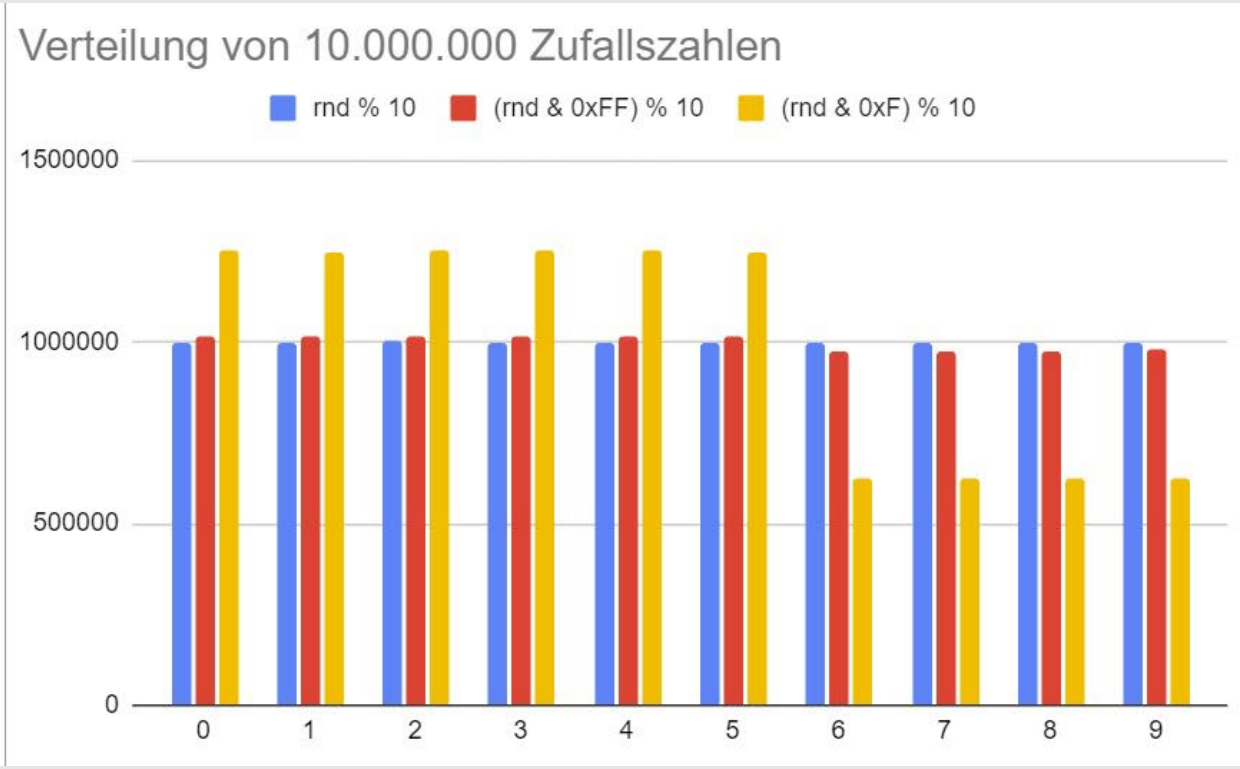
```
uint32_t rnd1 = *pRngDataReg;
uint32_t rnd2 = *pRngDataReg;
```

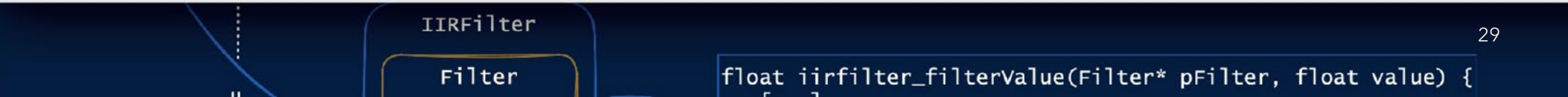
Zufallszahlentests
Testfunktion


```
Status_t testRNG(uint32_t observations, uint32_t m) {
    uint32_t* n = calloc(m, sizeof(uint32_t));
    if (n == NULL) {
        return Status_OutOfMemory;
    }
    for (int i = 0; i < observations; i += 1) {
        n[nextRand() % m] += 1;
    }
    Status_t status =
     ↪ equalDistChi2(n, m, (observations / m),
     ↪ chiSquared(m - 1, ChiSquaredAlpha_10_percent));
    free(n);
    n = NULL;
    return status;
}
```

Zufallszahltests
Auswertung

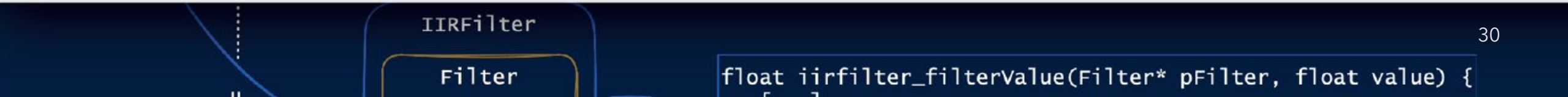




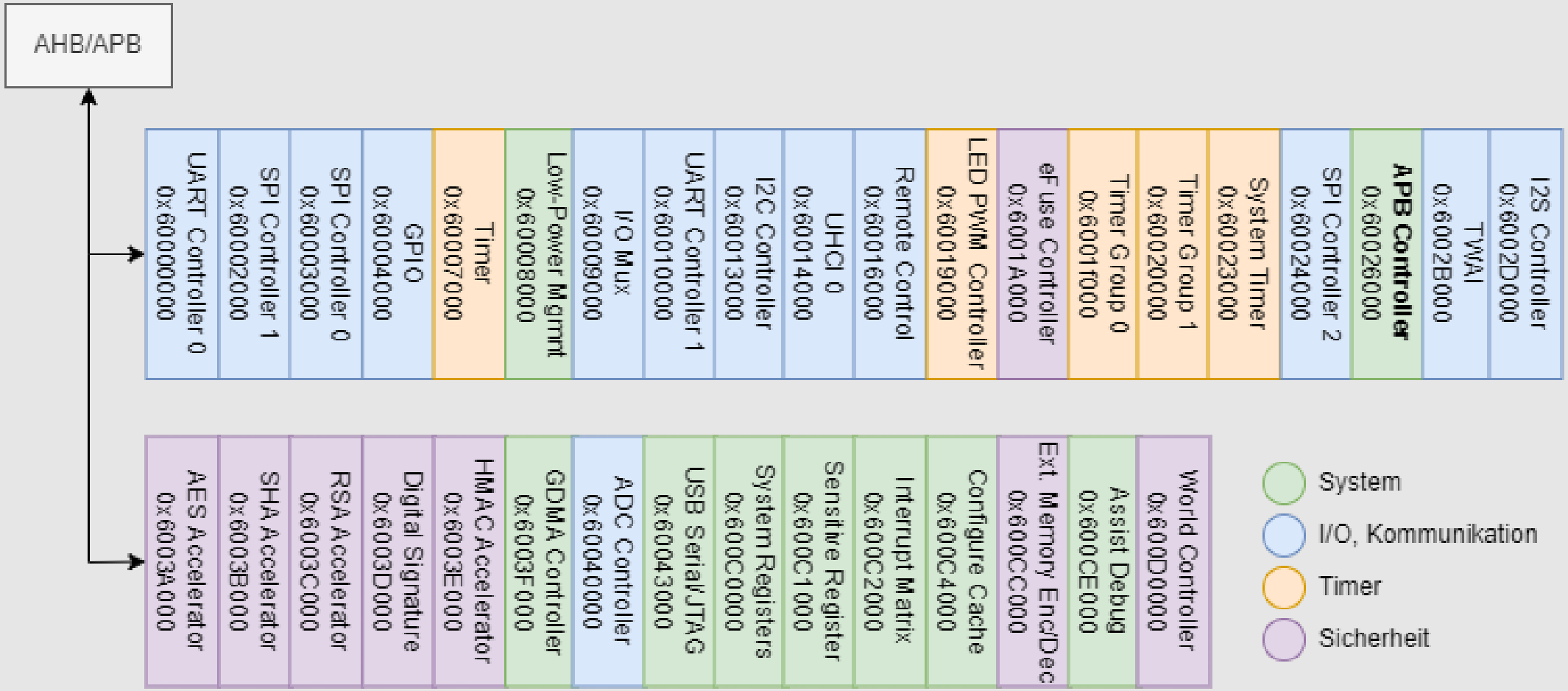


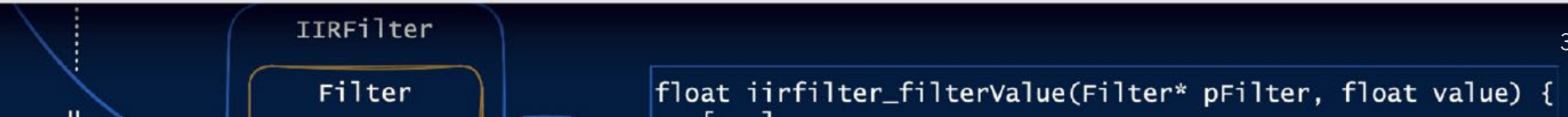
Hilfreiche Informationsquellen der Hersteller

- Verschiedene Arten von Information werden zur Verfügung gestellt
  - Data Sheet (Datenblatt)
  - Reference Manual, „Family Guide“
  - Errata
  - Application Notes
  - Development Boards und Demos
  - Software Frameworks und APIs
  - Material zur CPU
  - Compiler-Handbuch



Speicherlayout der Peripherie

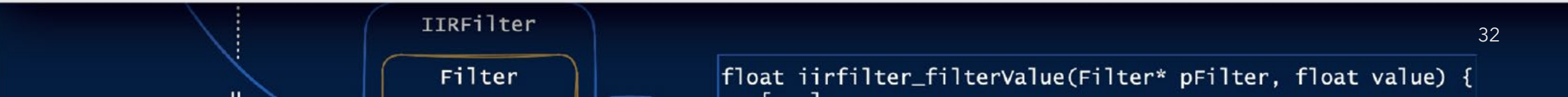




Bits als Schalter
Registerbeschreibung aus dem Datenblatt

RTC_CNTL_ANA_CLK_RTC_SEL	RTC_CNTL_FAST_CLK_RTC_SEL	RTC_CNTL_XTAL_GLOBAL_FORCE_NOGATING	RTC_CNTL_XTAL_GLOBAL_FORCE_GATING	RTC_CNTL_FOSC_FORCE_PU	RTC_CNTL_FOSC_FORCE_PD	RTC_CNTL_FOSC_DFREQ	RTC_CNTL_FOSC_FORCE_NOGATING	RTC_CNTL_XTAL_FORCE_NOGATING	RTC_CNTL_FOSC_DIV_SEL	(reserved)	RTC_CNTL_DIG_FOSC_EN	RTC_CNTL_DIG_FOSC_D256_EN	RTC_CNTL_DIG_XTAL32K_EN	RTC_CNTL_ENB_FOSC_DIV	RTC_CNTL_ENB_FOSC	RTC_CNTL_FOSC_DIV	RTC_CNTL_FOSC_DIV_SEL_VLD	RTC_CNTL_EFUSE_CLK_FORCE_NOGATING	RTC_CNTL_EFUSE_CLK_FORCE_GATING	(reserved)	
0	0	1	0	0	0	172	0	0	3	0	0	1	0	0	0	1	1	0	0	0	Reset
31	30	29	28	27	26	25	17	16	15	12	11	10	9	8	7	6	4	3	2	1	0

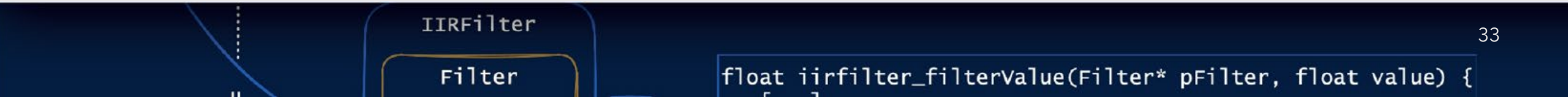
Register RTC_CNTL_CLK_CONF_REG (0x0070)



Bits als Schalter
Registerbeschreibung aus dem Datenblatt


```
#define LOWPOWER_MGR_BASE 0x60008000
#define RTC_CNTL_CLK_CONF_REG 0x70

volatile uint32_t* pRtcCntlClkConfReg =
    ↪ (volatile uint32_t*)
    ↪ (LOWPOWER_MGR_BASE | RTC_CNTL_CLK_CONF_REG);

inline void switchOnRtc20MClk() {
   *pRtcCntlClkConfReg = 0x00000400; // set Bit 10
}
```

Setzt Bit 10, löscht aber alle andern Bits!



Bitmaskierung
Klassische Aussagenlogik

Ein Satz hat eine Aussage

A_1: Ein KiB sind 1024 Byte $\Rightarrow true$

A_2: 5 ist durch 2 teilbar $\Rightarrow false$

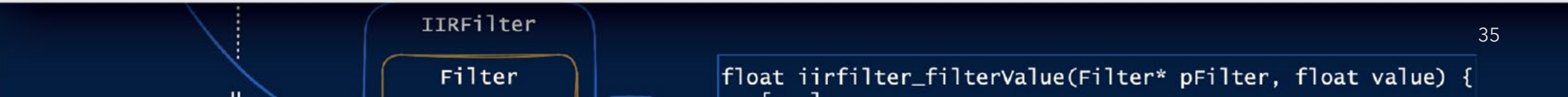
Verknüpfung

$$\underbrace{Ein\ KiB\ sind\ 1024\ Byte}_{true} \wedge \underbrace{Ein\ MiB\ sind\ 1024^2\ Byte}_{true} \Rightarrow true$$

$$\underbrace{Ich\ habe\ Kleingeld\ dabei}_{true} \wedge \underbrace{Ich\ habe\ nur\ Scheine\ dabei}_{false} \Rightarrow false$$

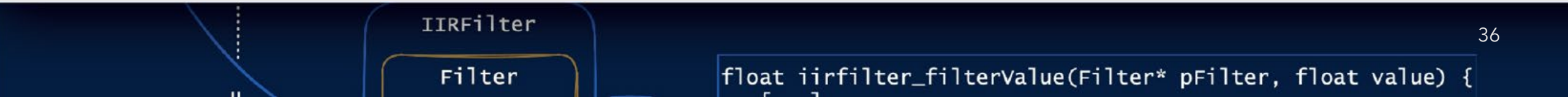
Bitmaskierung
Wahrheitstabelle der Junktoren

A	B	$A \wedge B$	$A \vee B$	$A \oplus B$
$false$	$false$	$false$	$false$	$false$
$false$	$true$	$false$	$true$	$true$
$true$	$false$	$false$	$true$	$true$
$true$	$true$	$true$	$true$	$false$



Bitmaskierung
Bitweise Operatoren in C, Beispiel

Statement	Bitmuster							
	b_7	a_6	a_5	a_4	a_3	a_2	a_1	a_0
`uint8_t a = 0xCA`	1	1	0	0	1	0	1	0
`uint8_t b = 0x43`	0	1	0	0	0	0	1	1
`a & b`	0	1	0	0	0	0	1	0
`a | b`	1	1	0	0	1	0	1	1
`a^b`	1	0	0	0	1	0	0	1
`~a`	0	0	1	1	0	1	0	1



Bitmaskierung
Bits setzen per OR

Bitweises OR mit der Maske der zu Setzenden Bits

```
*pRtcCntlClkConfReg |= 0x00000400 // set Bit 10
```

$$0000010000000000_{bin} = 0x0400$$

Define der Maske mit Shift
Statt hartcodiertem Wert

```
#define RTC_CNTL_DIG_FOSC_EN_BIT  10
#define RTC_CNTL_DIG_FOSC_EN
    ↪       (0x1 << RTC_CNTL_DIG_FOSC_EN_BIT)
```

Bitmaskierung
Dereferenzierung im Define


```
#define rtcCntlClkConfReg
    ↪ *((volatile uint32_t*)
    ↪ (LOWPOWER_MGR_BASE | RTC_CNTL_CLK_CONF_REG))
```

Einfachere Handhabung

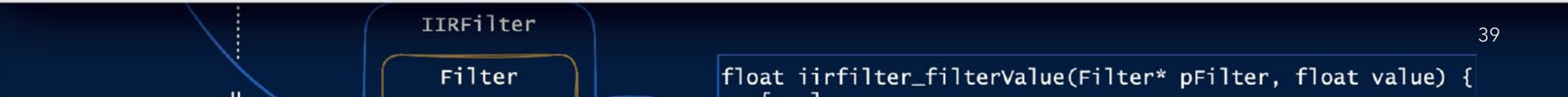
```
rtcCntlClkConfReg |= RTC_CNTL_DIG_FOSC_EN
```

Bitmaskierung
Bits löschen und toggeln

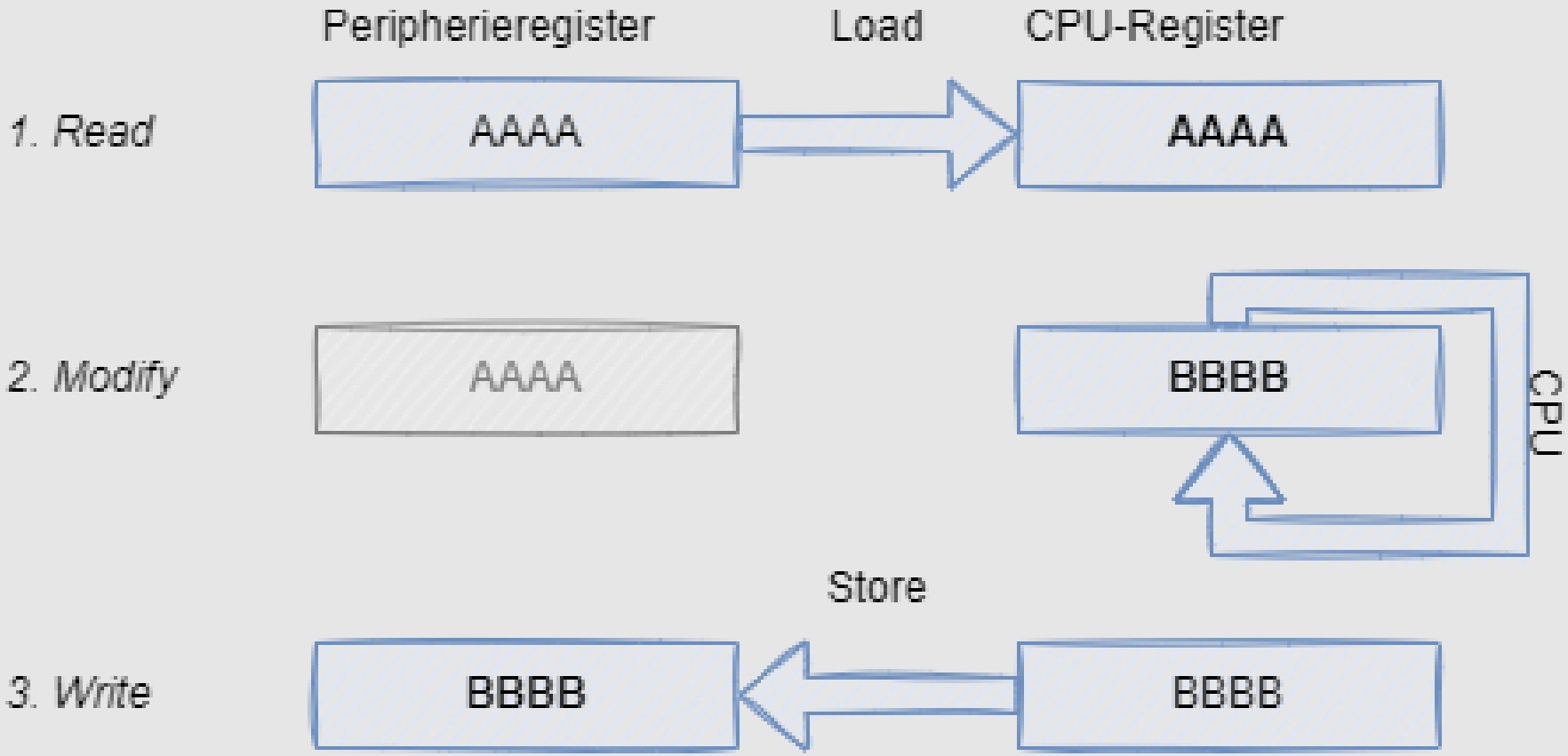
Löschen des Bits durch AND mit negierter Maske

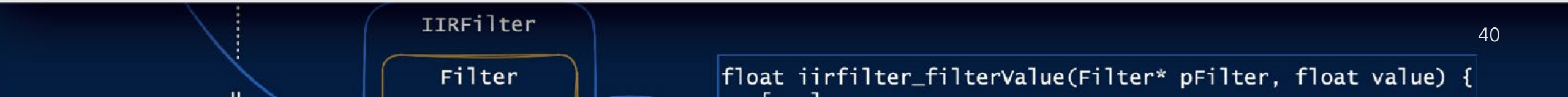
```
rtcCntlClkConfReg &= ~RTC_CNTL_DIG_FOSC_EN
```

- Per XOR werden die Bits der Maske „getoggelt“
  - ‚0‘-Bits werden auf ‚1‘ gesetzt
  - ‚1‘-Bits werden auf ‚0‘ gesetzt



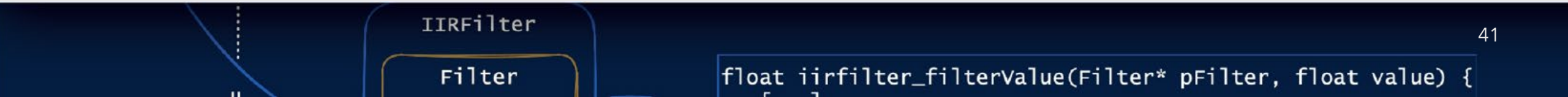
Registerzugriff
Read-Modify-Write





Bitmaskierung
Gleichzeitige Änderung mehrerer Bits


```
REG |= (BIT5 | BIT9 | BIT28); // Bits 5,9,28 setzen
REG &= (BIT5 | BIT9 | BIT31); // Bits 5,9,28 löschen
REG ^= (BIT5 | BIT9 | BIT31); // Bits 5,9,28 toggeln
```



Bitmaskierung
Mehrere Bits überscheiben


```
#define RTC_CNTL_FOSC_DFREQ_BIT 17
#define MASK_8BIT ((1 << 8) - 1)
#define RTC_CNTL_FOSC_DFREQ_MASK
    ↪ (MASK_8BIT << RTC_CNTL_FOSC_DFREQ_BIT)

void setRtcFOscDFreq(uint8_t dfreq) {
    uint32_t reg = rtcCntlClkConfReg;
    reg &= ~RTC_CNTL_FOSC_DFREQ_MASK;
    reg |= (dfreq << RTC_CNTL_FOSC_DFREQ_BIT);
    rtcCntlClkConfReg = reg;
}
```

Alle Bits des Bereichs löschen

Bits setzen